# Seminar 4

**Object-Oriented Design, IV1350**

Mo Wang mowang@ug.kth.se

2026-04-29

**Project Members:**
[Mo Wang, mowang@ug.kth.se]

## Declaration:

By submitting this assignment, it is hereby declared that all group members listed above have contributed to the solution. It is also declared that all project members fully understand all parts of the final solution and can explain it upon request.

It is furthermore declared that no part of the solution has been copied from any other source (except for resources presented in the Canvas page for the course IV1350), and that no part of the solution has been provided by someone not listed as a project member above.

## 1 Introduction

This report focuses on exception handling and design pattern implementation in the *Repair Electric Bike* system. The purpose of Seminar 4 is to extend the existing implementation with exceptions for error handling and to refactor the code using design patterns. In addition, unit tests are added to verify correct behavior in error-handling scenarios. The implementation follows the principles and guidelines described in Chapter 8 (exception handling) and Chapter 9 (design patterns) of *A First Course in Object-Oriented Development* [1].

Exception handling is used to manage both business logic violations and internal system failures. When a failure occurs in the model or integration layer, an exception is thrown and propagated to higher layers in the layered MVC-architecture. Exceptions are handled in the view layer, where a user-friendly error message is presented. For internal failures, detailed error information is also logged to a file for developers, according to the guidelines in Chapter 8 of the course literature. This approach avoids silent failures and error information propagation through return values.

In addition to exception handling, design patterns are applied to address recurring design problems in the system. In particular, the Observer pattern is used to enable

active notification of technicians and receptionists when a repair order is updated. This reduces coupling between system components and improves separation of concerns by decoupling update logic from presentation and logging responsibilities.

Throughout the implementation of exception handling and design patterns, automated unit testing is carried out in parallel with development. For each development iteration, corresponding test cases are added or updated to verify correct behavior. This continuous testing improves code quality and helps detect incorrect behavior early in the development process.

Seminar 4 consists of two main tasks: implementing exception handling according to the best practices described in Chapter 8 of the course book, and applying design patterns to improve system structure and flexibility, with particular focus on the Observer pattern and the introduction of additional design patterns for the same purpose.

Alongside the implementation, unit testing is performed using the JUnit framework, with particular focus on verifying exception-handling behavior. Tests ensure that exceptions are thrown under the correct conditions, that no unintended state changes occur after failures, and that error scenarios are handled consistently, as recommended in Chapter 8 of the course literature.

## Task 1: Exception Handling

The first task is to extend the Repair Electric Bike system with exception handling. The implementation must fulfill the following requirements:

- Exceptions are used to handle alternative flow 5a, where a searched customer phone number does not exist in the customer registry. They are also used to simulate database failures by throwing an exception when searching for a specific hard-coded item identifier.
- Exceptions must follow best practices described in Chapter 8 of the course book:
  - Appropriate choice between checked and unchecked exceptions.
  - Exceptions are defined at the correct abstraction level.
  - Exception classes are named after the error condition.
  - Exceptions include relevant information about the error.
  - Built-in functionality from `java.lang.Exception` is utilized.
  - Javadoc comments are written for all exception classes.
  - The system state must remain unchanged if an exception is thrown.
  - Users are notified with meaningful error messages.
  - Developers are notified through error logging to a file.
- When an exception is caught in the view layer, an informative error message is displayed to the user.
- Severe errors are logged to a file for debugging and maintenance purposes.

- Unit tests are implemented to verify exception handling behavior.

### Task 2: Design Patterns

The second task is to apply design patterns to improve the system design.

- The Observer pattern is implemented for active notification when a repair order is updated.
- A new view, `RepairOrderView`, prints updated repair orders to `System.out`.
- A second observer implementation, `RepairOrderLogger`, logs repair order updates to a file.
- Both observer implementations must implement the same observer interface and must not call the controller or other system components directly.
- For higher grade requirements, at least two additional Gang of Four (GoF) design patterns are implemented, excluding Observer and Template Method.
- The chosen patterns must be properly motivated and integrated into the system architecture.

### Unit Testing

- Unit tests are implemented continuously using the JUnit framework.
- Tests specifically verify exception handling and error scenarios.
- Tests ensure that exceptions are thrown under the correct conditions and that system state remains consistent.

## 2 Method

This chapter describes both the development process for implementing error handling using exceptions as well as applying design patterns in the code. Implementations are carried out while sticking to the code guidelines. Additionally, unit tests are written after each small implementation to ensure code correctness and quality.

### 2.1 Exceptions as error handling

For error handling, exceptions are used exclusively for representing and managing error conditions. The exception mechanism is dedicated to raising, propagating, and handling errors, ranging from business-rule violations to internal program failures. Error information is carried by exceptions, while return values are reserved for successful execution and non-error business data.

By avoiding error handling through return values, a clear separation is maintained between normal business logic and error-handling logic. This separation improves readability and prevents error-handling concerns from being mixed with normal control flow.

**Checked and unchecked exceptions** In terms of errors, there are two different error types for two exception types: business-rule violations and internal program failures.

Business-rule violations represent situations where a business rule is violated, for example searching for a non-existent customer or providing malformed input. These errors are expected to occur during normal execution and are therefore thrown as checked exceptions. Checked exceptions are declared in method signatures and documented in Javadoc, making the possible error conditions explicit to callers and enforcing handling at compile time.

Internal program failures represent technical problems or programming errors, such as database connection failures or unexpected runtime conditions. These errors cannot be meaningfully handled within the business logic and are therefore thrown using unchecked exceptions. Unchecked exceptions are not declared in method signatures and are logged for diagnostic purposes. When these exceptions cross architectural layer boundaries during propagation, they may be wrapped in higher-level exceptions to preserve the abstraction level of each layer, as described in Chapter 8 of the course literature.

**Exception Design Principles** Exception classes are designed to follow naming conventions. Each exception is named after error condition using descriptive names ending with `Exception`. Instead of creating many highly specific exception classes, a small number of well-defined exceptions were used, while detailed context was included in the exception message. Additionally, exceptions contain a descriptive message intended for developers, while avoiding exposure of internal implementation to user interface. This makes exception error messages useful and easier to debug the code.

In the layered MVC architecture, low-level unchecked exceptions originating from the integration layer are not propagated directly to the view. Instead, when appropriate, they are translated into higher-level checked exceptions, such as `FailedOperationException`, which represent business-relevant failure outcomes at the user level and can be meaningfully handled by the view. This ensures that the view layer only handles domain-relevant exceptions while preserving abstraction boundaries.

Objects must not change their internal state when an exception is thrown. Operations that do not inherently alter state, such as searching for a customer using the `find` method in the `CustomerRegistry`, can throw exceptions without side effects. In this case, a `NotFoundCustomerException` is thrown whenever the registry does not contain the specified telephone number.

For state-changing operations, input parameters are validated before any state changes occur and operations that may fail are executed before modifying the object state. In situations where this is not feasible, temporary variables are used to ensure that updates are only applied after all checks have passed. These strategies guarantee that the system remains in a consistent state even when errors occur.

**Writing Javadoc for exceptions** Additionally, methods that throws exceptions are documented using Javadoc `@throws`, which makes error condition explicit. The condition where the exception is thrown is documented clearly and serves as a guide to users of the method as well as keeping the developer to define the purpose of the exception. For instance, the method `find` in `CustomerRegistry` class finds a customer using a telephone number. If the telephone number does not correspond to any customer in the registry, a `NotFoundCustomerException` is thrown and propagated through the layer. The condition is stated in the method `find`, as well as the methods that propagate the exceptions.

**Notify user** Error messages for users are handled in the view layer. When an exception reaches the view, it is caught and converted into a user-friendly error message, which is then displayed. These messages are designed to be clear and informative about business rule violations, without exposing internal system details. This avoids duplication and prevents lower-level components from directly interacting with the user interface.

**Notify developers** Notifying error messages to developers is handled through a dedicated logging mechanism. When unchecked exceptions occur, they are logged to a file along with detailed diagnostic information, including stack traces, which allows developers to trace and debug errors effectively. The logging mechanism is implemented in a separate utility component to maintain separation of concerns. However, business-rule exceptions, such as malformed input, are not logged since they are expected execution behavior.

**Exception unit tests** Exceptions are unit tested to verify correct behavior in both normal and failure scenarios. Tests ensure that no exceptions are thrown during valid execution and that the appropriate exception types are thrown under failure conditions. They also verify that exception messages contain useful information and that exceptions are correctly propagated across layers, enabling proper user notification and logging when handled.

## 2.2 Design Patterns

Design patterns are introduced to improve the structure, flexibility, and maintainability of the system by addressing recurring design problems identified during implementation. According to the guidelines in Chapter 9 of the course literature, patterns are applied when existing solutions contains tight coupling, low flexibility, or require modifications in multiple places when functionality is extended. In such cases, suitable GoF design patterns are applied to refactor the code and clearly separate responsibilities.

The pattern selection process starts by analyzing responsibilities and dependencies in the existing system. Code sections that required frequent modification or where changes in one component affected multiple other components were identified as candidates for refactoring. Patterns were chosen based on their ability to reduce coupling, increase cohesion, and encapsulate variation, according to the principles described in Chapter 9.

Several design patterns are introduced, each selected to address a specific design problem identified during system evolution.

**Polymorphism** Polymorphism plays a central role in the application of design patterns. Interfaces are introduced to ensure that clients depend only on abstractions rather than concrete implementations, allowing behavior to be substituted without modifying existing code. This is particularly important for patterns such as Observer and Strategy, where multiple interchangeable implementations must conform to a common interface.

**Observer pattern** The Observer pattern is implemented to address the requirement for automatic notification of `RepairOrderView` when a repair order is updated. Instead of having direct interaction between components for updating the repair order status, which leads to higher coupling, the observer pattern is introduced. By applying the Observer pattern, dependent views can be notified of state changes without the observed object depending on concrete implementations. This supports extensibility and reduces coupling between update logic and presentation concerns.

**Singleton pattern** The Singleton pattern is applied to classes that is only allowed to be instantied once. These components are required to maintain a single, consistent state throughout the application lifecycle, and creating multiple instances could lead to resource conflicts or inconsistent behavior.

To enforce single instantiation, such classes are designed with a private constructor and a single static instance that is initialized eagerly. Access to the instance is provided through a public static method. This design ensures that all parts of the system interact with the same resource instance while preventing accidental instantiation from other components.

The use of the Singleton pattern simplifies resource management and enforces a clear lifecycle for shared infrastructure components.

**Strategy pattern** The Strategy pattern is applied in situations where alternative algorithms must be selected dynamically based on runtime conditions. During development, pricing logic was identified as behavior that varied depending on customer membership status and repair history. Embedding conditional pricing logic directly into domain classes would have reduced readability and made future extensions more difficult.

To address this, pricing behavior was encapsulated into separate strategy classes implementing a common interface. Each strategy defines a distinct pricing algorithm, allowing different calculation rules to be substituted transparently. The repair order delegates pricing computation to a strategy instance instead of containing the pricing logic itself.

The selection of the concrete strategy is performed through a factory, enabling the appropriate pricing behavior to be chosen based on customer information at runtime. This design isolates variation, supports extension with new pricing rules, and avoids modifying existing business logic when pricing requirements change.

**Factory pattern** The Factory pattern is applied to centralize object creation logic and decouple the system from concrete class implementations. During initialization, certain components such as data handlers must be instantiated based on configuration or runtime parameters, rather than being hardcoded.

To support this, object creation is delegated to factory classes that encapsulate the instantiation logic. The factory determines which concrete implementation to create based on provided parameters and returns an interface type to the caller. This allows the rest of the system to operate independently of specific implementations.

Using the Factory pattern improves flexibility, simplifies configuration-based initialization, and prevents the spread of instantiation logic across multiple components.

# 3 Result

By extending the current implementation of the Repair Electric Bike system with exception handling, errors are handled using dedicated exception classes and propagated through the system layers until they reach the view..

Design patterns are also integrated to improve the code quality and program structure. For instance, the Observer pattern is implemented to notify an interactive `RepairOrderView` when repair orders are updated, which allows multiple views to react and update automatically without direct dependencies on the model or the controller.

In addition, logging functionality has been introduced to track internal program failures only, making debugging easier, while user-friendly error messages are presented in the view. Business logic violations are displayed only in the view and are explicitly excluded from logging, since the internal program still produces correct behavior.

Alongside the implementations, unit tests are written to verify both normal execution and error-handling behavior.

The complete source code for this seminar is available in the public Git repository at: `https://github.com/mo-mosverkstad/java-learning/tree/main/maven-projects/ElectricBikeRepair/` [2].

## 3.1 Error handling using exceptions

In terms of error handling using exceptions, custom exception classes are extended from the `Exception` base class to cover different groups of error conditions. Business-rule violations are represented using checked exceptions, while internal failures are represented using unchecked exceptions.

The system distinguishes between two categories of exceptions: checked exceptions for business-rule violations and unchecked exceptions for internal program failures. Business-rule exceptions are thrown in the model or utility layers and are propagated unchanged through the controller to the view, where they are presented as user-friendly error messages. Unchecked exceptions originate in the integration layer and represent technical failures; these are caught in the controller layer, logged, and translated into a higher-level checked exception before being propagated to the view. As a result, only business-relevant exceptions reach the view layer.

### 3.1.1 Checked exception handling - InvalidTelephoneNumberException

The `InvalidTelephoneNumberException` exception arises when a telephone number is malformed and therefore cannot be read and parsed. Since the exception represents a violation of a business rule, it is implemented as a checked exception and extends `Exception`. The error message contains the malformed telephone number to inform the user.

The checked exception `InvalidTelephoneNumberException` is defined to represent the error condition where a telephone number has an invalid format with the specific invalid input, following the naming conventions in Chapter 8. The code can be found in `.../util/InvalidTelephoneNumberException.java` [2].

The exception `InvalidTelephoneNumberException` is thrown in `TelephoneNumber` class constructor, located in utility layer. The `TelephoneNumber` takes responsibility of normalizing arbitrary formatted telephone number into E-164 format. When a telephone number is beyond recognition of any predefined format, the telephone number cannot be processed and therefore an exception `InvalidTelephoneNumberException` is thrown, whereas the logic is visualized in the listing. The implementation can be found in `.../util/TelephoneNumber.java` [2].

The methods that propagate the exception must also declare the exception after the `throws` keyword, which specifies the thrown exceptions. Since the methods `findRepairOrder` as well as the internal private function `findRepairOrderIds` depends on the `TelephoneNumber` class, which throws InvalidTelephoneNumberException.

Additionally, in `ReceptionistController` in controller layer, the methods `searchCustomer` and `createRepairOrder` both propagates the exception `InvalidTelephoneNumberException`.

The method `searchCustomer` propagates checked business-rule exceptions and translates unchecked technical failures into a higher-level checked exception, as shown later in the unchecked exception handling example.

The method `createRepairOrder` invokes `searchCustomer`, which may propagate the checked exception `NotFoundCustomerException` when the specified customer does not exist. The listing also illustrates how unchecked technical exceptions are caught locally and translated into a higher-level checked exception before being propagated to the caller.

When the checked exception `InvalidTelephoneNumberException` reaches view, it is caught alongside with other checked exceptions. The internal methods that proceeds each hardcoded action are defined in `proceedReceptionPreparationActions`, `proceedTechnicianDiagnosticActions`, and `proceedReceptionConfirmationActions` passes all their checked exception to the larger `proceedActions` method where the unchecked exceptions are all handled collectively. The exception information, including error message, is extracted and displayed to the view. Since the view is simple, it only extracts the error message from the exception and printing it into standard output. After displaying the error message, no further workflow steps are executed.

Listing 1: View.proceedActions catching exceptions and displaying error messages

```
1 /**
```

```
 * Runs the full repair workflow: reception preparation, technician diagnostics
    , and reception confirmation.
 *
 * @param telephoneNumber the customer's telephone number
 * @param bikeSerialNumber the serial number of the broken bike
 */
public void proceedActions(String telephoneNumber, String bikeSerialNumber) {
    try{
        proceedReceptionPreparationActions(telephoneNumber, bikeSerialNumber);
        proceedTechnicianDiagnosticActions(telephoneNumber);
        proceedReceptionConfirmationActions(telephoneNumber);
    } catch (NotFoundCustomerException | InvalidTelephoneNumberException |
    NoExistedRepairOrderException e) {
        System.out.println(ERROR_PREFIX + e.getMessage());
    } catch (FailedOperationException e) {
        System.out.println(ERROR_PREFIX + e.getMessage());
        LogHandler.getLogger().logException(e);
    }
}
```

### 3.1.2 Unchecked exception handling - ResourceNotFoundException

Unchecked exceptions are propagated through the layers without declaring it after `throws` keyword. The unchecked exception `ResourceNotFoundException` is thrown when a data resource cannot be found. The exception contains the error message with the resource name that cannot be found. The implementation of the unchecked exception definition can be found in `.../data/ResourceNotFoundException.java` [2].

The exception is thrown during initialization of a resource handler of `DatabaseHandler` in the method `initDataHandler`. The exception is propagated further through methods such as `loadDiagnosticTasks` of `DiagnosticReport` and `initRegistry` of `CustomerRegistry`. In these methods, the exception is not declared, but implicitly thrown.

Eventually, all unchecked resource exceptions are the controller layer and are converted into a checked exception `FailedOperationException`, which can be handled and displayed in view layer. The methods `searchCustomer` and `createRepairOrder`, for example, catches all possible unchecked exceptions and wraps them into a `FailedOperationException`, which is thrown to the view caller. The method `searchCustomer` shows the wrapping process of multiple unchecked exceptions such as `ResourceNotFoundException`, in the catch block of the method. The error message is also logged as well using the `LogHandler`

Listing 2: ResourceNotFoundException wrapped to FailedOperationException

```
/**
 * Searches for a customer by telephone number.
 *
 * @param telephoneNumber the customer's telephone number (any Swedish format)
 * @return the found customer DTO
 * @throws NotFoundCustomerException if the phone number does not correspond to
     a customer
```

```
7   * @throws InvalidTelephoneNumberException if the telephone number is empty or
        invalid
8   * @throws FailedOperationException if an underlying infrastructure error
        occurs, such as a database or file system failure
9   */
10  public CustomerDTO searchCustomer(String telephoneNumber) throws
        NotFoundCustomerException, InvalidTelephoneNumberException,
        FailedOperationException{
11      try {
12          // search customer operation
13      } catch (ResourceNotFoundException | InvalidResourceURIException |
        CannotReadFileException | CannotWriteFileException | NoDatabaseException
        exception){
14          LogHandler.getLogger().logException(exception);
15          throw new FailedOperationException("Unable to search customer due to an
         internal system error", exception);
16      }
17  }
```

### 3.1.3 Log handling for exceptions

Reporting error messages is handled by a special and dedicated utility class `LogHandler`. The class is responsible for writing formatted logs to a specified program file `ebikerepair.log`.

Since the class is only instantiated once and the resource is never duplicated, the `singleton pattern` is applied. This is accomplished by making the constructor private as well as keeping its only singleton instance as a static class member, along with its desired log filepath. The instance is retrieved by the static method `getLogger` before the method `logException` can be called for writing the error message to the log.

During logging, the `logException` method is called from the controller layer whenever an internal unchecked exception is caught. The method takes an exception class of any type and extract out the error message. Unchecked technical exceptions are logged together with detailed diagnostic information, including exception type, message, cause, and stack trace, while business-rule violations are excluded from logging, since they represent expected execution scenarios. The detailed implementation of `logException` can be found in `.../util/LogHandler.java` [2].

## 3.2 Unit testing for handling errors and exceptions

Unit tests are written to verify that each method that throws or propagates exceptions produces correct behavior in different conditions.

### 3.2.1 Unit tests for checked exceptions

In terms of testing methods that throws checked exception, positive test cases are expected to not throw any exception while negative test cases are expected to produce a specified exception. In this case, the `TelephoneNumber` constructor is tested

in the `TelephoneNumberTest` class. It verifies that valid phone numbers are read and interpreted without throwing, and that malformed formats must result into an `InvalidTelephoneNumberException` that is thrown. The test cases are checked using the method `assertThrows` during a test case where a specified exception is thrown. If a test case is not expected to throw any exception, the method `assertDoesNotThrow` is used instead. The test implementation is found in `.../util/TelephoneNumberTest.java` [2].

The `searchCustomer` method in `ReceptionistController` class is tested in the `ReceptionistControllerTest` class. Since the `searchCustomer` method calls the `TelephoneNumber` that throws the specific exception`InvalidTelephoneNumberException` and other methods that throws similar methods, it is therefore tested and verified. It verifies that an existing customer is found without throwing, that `NotFoundCustomerException` is thrown for a non-existing customer, and that `InvalidTelephoneNumberException` is propagated for malformed input. The test implementation can be found in `.../controller/ReceptionistControllerTest.java` [2].

### 3.2.2 Unit tests for unchecked exceptions

For unchecked exceptions, the unit tests are also written in same structure as the checked exceptions. The only difference is that the unchecked exceptions are not declared in public API. For instance, the exception `ResourceNotFound` exception is expected to be thrown when a non-existent resource is tried to be accessed in `initDataHandler` method, which can be found in `.../data/DatabaseHandlerTest.java` [2].

### 3.2.3 Unit tests for log handler

Additionally, the unit tests for log handler are also written. For testing logging, an exception is created whereas a unique message is written inside it. The message is logged afterwards into a file which is later read to retrieve out the value and compare with the original error message to verify the correctness of the logging functionality.

Listing 3: Unit test for writing logging

```
@Test
void testLogExceptionWritesToFile() throws IOException {
    LogHandler logger = LogHandler.getLogger();
    String uniqueMessage = "Test exception " + System.nanoTime();
    Exception testException = new Exception(uniqueMessage);

    logger.logException(testException);

    File logFile = new File("ebikerepair.log");
    assertTrue(logFile.exists());
    String content = Files.readString(logFile.toPath());
    assertTrue(content.contains(uniqueMessage));
    assertTrue(content.contains("[ERROR]"));
}
```

## 3.3 Sample run printouts

The sample run printouts for exception handling and error reporting concerns both checked and unchecked exception. Checked exceptions result in user-friendly error messages displayed in the view, while unchecked exceptions indicate technical failures and are logged to a file for developer inspection, according to Chapter 8 of course literature.

### 3.3.1 Checked exception printouts

When a checked exception occurs due to a business-rule violation, such as an invalid telephone number or a non-existent customer, the exception is propagated to the view layer without being logged. The view ultimately catches the exception and displays a user-friendly and informative error message to the user. The purpose is to provide a clear feedback about the incorrect input or invalid business condition, while the program continues.

The listing below shows the message of a sample printout concerning a malformed telephone number. Since it concerns business logic violation, the error message is designed to notify the user about the invalid input as a feedback.

Listing 4: Checked exception as error message

```
Error: The phone number 11707654322 is not a valid phone number
```

### 3.3.2 Unchecked exception printouts - writing logs

Unchecked exceptions that contains internal program failure are logged to a file `ebikerepair.log`, which simplifies debugging and therefore require additional diagnostic information for developers. The log contains two messages for one unchecked exception: one for the unchecked exception that is caught in controller layer and the lower one for the unchecked exception being wrapped into a checked exception in `FailedOperationException`, before being logged.

For instance, the listing below shows that a missing database, which is a internal failure, is displayed in two log messages. The first log entry corresponds to the original internal failure (a missing database), and the second entry shows the higher-level checked exception (`FailedOperationException`) that wraps the original cause before being propagated to the view. This shows how abstraction boundaries are preserved while still retaining detailed diagnostic information in the log.

Listing 5: Unchecked exception and wrapped checked exception captured in log file

```
[May 9, 2026, 11:49:10 AM] [ERROR] se.ebikerepair.data.NoDatabaseException: No database customers
    found or connected
  Stack trace:
    at se.ebikerepair.data.DatabaseHandler.initDataHandler(DatabaseHandler.java:35)
    at se.ebikerepair.integration.CustomerRegistry.initRegistry(CustomerRegistry.java:37)
    at se.ebikerepair.integration.CustomerRegistry.find(CustomerRegistry.java:51)
    at se.ebikerepair.controller.ReceptionistController.searchCustomer(ReceptionistController.java
     :55)
    at se.ebikerepair.view.View.proceedReceptionPreparationActions(View.java:61)
```

```
    at se.ebikerepair.view.View.proceedActions(View.java:49)
    at se.ebikerepair.startup.Main.main(Main.java:32)
    at org.codehaus.mojo.exec.ExecJavaMojo$1.run(ExecJavaMojo.java:279)
    at java.base/java.lang.Thread.run(Thread.java:1583)

[May 9, 2026, 11:49:10 AM] [ERROR] se.ebikerepair.controller.FailedOperationException: Unable to
     search customer due to an internal system error
  Caused by: se.ebikerepair.data.NoDatabaseException: No database customers found or connected
  Stack trace:
    at se.ebikerepair.controller.ReceptionistController.searchCustomer(ReceptionistController.java
     :58)
    at se.ebikerepair.view.View.proceedReceptionPreparationActions(View.java:61)
    at se.ebikerepair.view.View.proceedActions(View.java:49)
    at se.ebikerepair.startup.Main.main(Main.java:32)
    at org.codehaus.mojo.exec.ExecJavaMojo$1.run(ExecJavaMojo.java:279)
    at java.base/java.lang.Thread.run(Thread.java:1583)
```

The listing below shows a user-friendly message of a sample printout concerning a internal system failure. The error cause is mentioned without going into detail of the internal mechanism.

Listing 6: Checked exception as error message

```
Error: Unable to search customer due to an internal system error
```

## 3.4 Design pattern designs

Design patterns are implemented in the system to improve code quality by adding flexibility while reducing coupling between classes.

### 3.4.1 Observer design pattern

The Observer pattern is implemented to notify all interactive observers when a repair order is updated. A common observer interface is defined, which is implemented by multiple observer classes.

The class `RepairOrder` acts as the subject (the observed object) and maintains a list of registered observers, while `RepairOrderView` and `RepairOrderLogger` are concrete observers that both implements `RepairOrderObserver`. Whenever the repair order is updated, all registered observers are notified automatically by calling a notify method.

Two observer implementations are provided. The class `RepairOrderView` displays updated repair order information to the user via `System.out`. The class `RepairOrderLogger` writes the updated repair order information to a file for logging purposes. Both observers implement the same interface and are updated without directly interacting with the controller or other system components.

Observers are registered during view initialization and later associated with newly created repair orders.

The observer interface defines the public interface that all observer implementations must implement. The method `updateRepairOrder` is invoked when the observed repair order changes and acts as an entry point to notify an observer object.

Listing 7: RepairOrderObserver interface

```
1 public interface RepairOrderObserver {
2     public void updateRepairOrder(RepairOrderDTO repairOrderDTO);
3 }
```

The `RepairOrderView` class implements the observer interface and provides a user-facing representation of repair order updates. When notified, it prints a structured notification message to standard output, allowing users to be informed of changes in repair order status without explicitly requesting this information.

The `RepairOrderLogger` class provides an alternative observer implementation that records repair order updates to a file instead of presenting them to the user. This implementation shares the same notification interface and mechanism as the view observer, but directs output to a persistent log instead.

The `RepairOrder` class maintains its own list of observers, which is populated when a repair order is created. Whenever the internal state of the repair order changes, all registered observers are notified by calling the internal method `notifyRepairOrderObservers` that collectively notifies the observers with the current state represented as a data transfer object.

During state changes, the observer objects must be therefore notified. This is implemented by notifying the observers using the method `notifyRepairOrderObservers` for every method that alters the state of a `RepairOrder` object. In this case, the methods `updateProblem`, `accept` and `reject` modifies the internal fields of the object, and the notification is handled at the end of each procedure.

The controller is responsible for maintaining a collection of observers that should be notified of repair order updates. Observers are registered once and stored internally by the controller, allowing them to be used for observed objects in the business logic. The observer collection inside controller enables an observer interface between View and Controller layers while avoiding direct coupling between View and Model layer.

### 3.4.2 Singleton design pattern

The singleton design pattern allows a class only to be instantiated once, which is useful for database connections and log handling objects. This means it is important to ensure that only one shared instance of resource-handling components exists throughout the application lifecycle.

The pattern is designed by keeping the only instance of the class as its static final member, while keeping the constructor private to protect it from being instantiated accidentally. This ensures that only one instance of the class persists.

The listing below shows an example of a singleton class `DatabaseHandler`. The handler class keeps its only instance as `INSTANCE` which is automatically instantiated. The instance can only be accessed by calling the static method `getInstance`, while creating the second instance is impossible due to its private constructor.

Listing 8: `DatabaseHandler` implementation with singleton

```
1 public class DatabaseHandler{
```

```
    private static final DatabaseHandler INSTANCE = new DatabaseHandler();
    private String databaseName;

    /**
     * Returns the singleton DatabaseHandler instance.
     *
     * @return the single DatabaseHandler instance
     */
    public static DatabaseHandler getInstance() {
        return INSTANCE;
    }

    /**
     * No-arg constructor kept for reflection-based instantiation via {@link
    DataHandlerFactory}.
     */
    private DatabaseHandler() {}

    // Methods for database handling
}
```

### 3.4.3 Strategy design pattern

Additionally, the strategy design pattern is applied where algorithm can be dynamically replaced and interchanged. In this case, the strategy design pattern is applied for customer memberships and discounts, where the customer is entitled to a 15% discount every third bike repair. This means that non-membership customers and membership customers are priced differently using different pricing algorithms.

The pricing algorithm is written inside the method `calculateTotalCost` of a dedicated class. In this case, an interface `PricingStrategy` is defined to keep the method name and signature of `calculateTotalCost` consistent among pricing algorithm classes.

Listing 9: Pricing strategy interface

```
public interface PricingStrategy {
    PricingResult calculateTotalCost(RepairOrder repairOrder);
}
```

A default pricing strategy is provided for non-membership customers. The `StandardPricingStrategy` calculates the total repair cost based solely on diagnostic tasks and repair tasks, without applying any discounts. This strategy represents the baseline pricing behavior used when no membership rules apply. `.../model/StandardPricingStrategy.java` [2].

For membership customers, an alternative pricing strategy is implemented. The `MembershipPricingStrategy` applies a discount every third repair based on the customer's repair history. This strategy uses the same base cost calculation as the standard strategy, but conditionally applies a discount before computing the final price. `.../model/MembershipPricingStrategy.java` [2].

The selected pricing strategy is applied when the total repair cost is requested from a repair order. The repair order delegates the pricing calculation to a strategy instance obtained from a factory based on customer information to produce a suitable instance with algorithm, allowing the concrete strategy to be selected dynamically based on customer information without modifying the repair order logic. This means that the responsibility for selecting the appropriate pricing strategy is delegated to a factory, keeping the repair order independent of concrete pricing rules.

Listing 10: Applying pricing strategy in `RepairOrder`

```
/**
 * Returns the total cost from diagnostic tasks and repair tasks combined.
 *
 * @return the total cost
 */
public Cost getTotalCost() {
    return getPricingResult().getFinalCost();
}

public PricingResult getPricingResult(){
    PricingStrategy pricingStrategy = PricingStrategyFactory.create(
    getCustomerDTO());
    return pricingStrategy.calculateTotalCost(this);
}
```

### 3.4.4 Factory design pattern

Finally, the factory design pattern is useful in terms of initializing objects dynamically where the class of the object can be decided during runtime. A class name is stored as a string and passed into the `create` method, which uses Java reflection to dynamically dispatch and instantiate the desired class object. In this case, the `DataHandlerFactory` initializes a `DataHandler` object based on the class name stored in the string. This allows dynamic dispatching where the object initialization is determined during runtime, which adds flexibility during program initialization.

Listing 11: DataHandlerFactory that instantiates data handler object dynamically

```
/**
 * Factory for creating {@link DataHandler} instances by class name using
    reflection.
 */
public class DataHandlerFactory {

    private DataHandlerFactory() {}

    /**
     * Creates a DataHandler instance from the given fully qualified class name
     *
```

```
11      * @param className the fully qualified class name of the DataHandler
        implementation
12      * @return a new DataHandler instance
13      * @throws ClassNotFoundException if the class cannot be found
14      * @throws NoSuchMethodException if the class has no no-arg constructor
15      * @throws InstantiationException if the class cannot be instantiated
16      * @throws IllegalAccessException if the constructor is not accessible
17      * @throws InvocationTargetException if the constructor throws an exception
18      */
19     public static DataHandler create(String className) throws
        ClassNotFoundException, NoSuchMethodException, InstantiationException,
        IllegalAccessException, InvocationTargetException {
20         Class<?> classObject = Class.forName(className);
21         return (DataHandler) classObject.getConstructor().newInstance();
22     }
23 }
```

Errors during reflective instantiation are propagated to the caller and handled by the existing exception-handling mechanism.

### 3.5 Unit tests for design pattern mechanism

**Observer design tests** The Observer pattern is verified through the `RepairOrderView`, which acts as a concrete observer producing interactive notifications when a repair order changes state. The observer mechanism is tested by executing the full workflow of the bike repair system using the `proceedActions` method and capturing the printed output.

The test verifies that observer notifications are produced during the execution of the workflow by asserting the presence of characteristic notification keywords in the printed output. `.../view/ViewTest.java` [2]

**Singleton design tests** The Singleton pattern is tested using the `LogHandler` class, which is designed to only allow one single shared instance throughout the application. The tests verify that multiple calls to the `getLogger` method return the same instance and that the singleton instance is correctly initialized. `.../util/LogHandlerTest.java` [2]

**Strategy design tests** The Strategy pattern is tested by verifying that different pricing algorithms are applied depending on customer membership status and repair history. The tests confirm that non-membership customers receive no discount in `testNonMemberNoDiscount`, while membership customers receive a 15% discount on every third repair in `testMemberThirdRepairGets15`.

The correctness of strategy selection and execution is verified by asserting the base cost, discount amount, and final cost returned by the pricing calculation. Helper methods are used to construct repair orders with the required membership configuration and repair count for each test case. `.../model/RepairOrderTest.java` [2]

**Factory design tests** The Factory pattern is tested by verifying that the `DataHandlerFactory` creates the correct concrete data handler implementation based on the provided class name. Successful instantiation of known handler implementations is verified, while invalid class names are tested to ensure that appropriate exceptions are thrown. `.../data/DataHandlerFactoryTest.java` [2]

### 3.6 Sample run printouts of design patterns

The sample run printouts of the functionality based on design patterns includes interactive notification of repair order using observer pattern as well as a membership-based discounts using strategy pattern. The remaining two functionalities implemented by singleton and factory design pattern does not produce direct user-facing output, but improves instead the flexibility of the program design.

The listing below shows a sample run printout for a member customer `Astrid Johansson` during third bike repair. The repair order notification is shown deliminated by the line brackets `--- REPAIR ORDER MODIFICATION NOTIFICATION ---`, whereas it contains several detail regarding the repair order id and telephone number. Additionally, after two previous repairs in membership, the customer is entitled to a 15% discount of a repair order, which is shown in the `-- costs --` section of the printout receipt. The base cost, the deducted cost and the resulting total cost are all shown.

Listing 12: Sample printout of repair order receipt

```
8. Reception - Accepted order
--- REPAIR ORDER MODIFICATION NOTIFICATION (BEGIN) ---
THE REPAIR ORDER HAS BEEN MODIFIED : 399e9c4f-efcd-4a45-9b2b-2af4be4261df
PLEASE USE TELE NUMBER TO CHECK DETAILS: +46707654321 (Astrid Johansson)
--- REPAIR ORDER MODIFICATION NOTIFICATION -(END)- ---

**** Printing repair order FROM PRINTER ****
==================================
  Repair Order
==================================
  Order ID:       399e9c4f-efcd-4a45-9b2b-2af4be4261df
  Status:         Accepted
  Created:        Sun May 03 13:13:01 CEST 2026
  Est. Complete:  Thu May 07 13:13:01 CEST 2026
  -- Costs --
  base = 1175.00 SEK, discount = 176.25 SEK, finalCost = 998.75 SEK
  Total Cost:     998.75 SEK
**********************************
...
```

## 4 Discussion

This seminar focused on improving program structure and quality by implementing exception mechanism as error handling as well as applying design patterns. The imple-

mented solution is evaluated below using assessment criteria for Seminar 4, particularly on correctness, motivation, and architectural consequences.

## 4.1 Evaluation of Exception Handling

The exception-handling mechanism follows the nine guidelines described in Chapter 8 of the course literature and fulfills the requirements of Task 1.

Business-rule violations, such as invalid input or missing customer data, are represented using checked exceptions, for instance `InvalidTelephoneNumberException` and `NotFoundCustomerException`. These exceptions clearly represent expected error situations that may occur during normal execution and therefore require explicit handling by the caller. Declaring them in method signatures makes the error conditions explicit and enforces correct handling at compile time.

Internal program failures, such as missing resources or simulated database failures, are covered using unchecked exceptions. These failures are outside the control of the business logic and cannot be resolved meaningfully by the caller. Unchecked exceptions are therefore logged and wrapped into higher-level checked exceptions (e.g. `FailedOperationException`) when crossing architectural layer boundaries. This preserves abstraction levels and prevents technical details from leaking into the view layer, which is consistent with the separation principles described in Chapter 8.

When throwing errors, the system state is not altered. This is accomplished using input validation before modifying state or executing all error-prone operations have completed before state change operations. This ensures that errors during program execution do not alter the system state.

User notification and developer notification are clearly separated. Business-rule exceptions are presented as meaningful, user-friendly messages in the view layer and are not logged, since they represent normal execution paths. Technical failures are logged with full diagnostic information using a dedicated logging utility, while the user receives a general failure message. This separation follows the explicit recommendation in the course literature that business exceptions shall not be logged, while internal failures shall be.

Unit tests ensures intended behavior in both normal execution and error scenarios. Tests confirm that the correct exception types are thrown, that exception propagation works across layers, and that logging is only reserved for internal failures. This testing strategy aligns with the guidelines in both Chapter 7 and Chapter 8.

Overall, the exception-handling solution is consistent, well-motivated, and follows established best practices without mixing error handling with normal control flow.

## 4.2 Evaluation of Observer Pattern Usage

The Observer pattern is correctly implemented and motivated by shortcomings in the original system design. In the original version of the Repair Electric Bike system, both technicians and receptionists had to actively request updated repair order information, leading to unclear responsibility for update propagation.

By introducing the Observer pattern, repair orders actively notify its listeners when the state changes. The class `RepairOrder` acts as the observed subject, while `RepairOrderView` and `RepairOrderLogger` are concrete observers. This is a faithful implementation of the GoF Observer pattern as described in Chapter 9.

Observers implement a common interface and are notified exclusively through the observer mechanism. They do not call the controller or any other system component directly, which prevents cyclic dependencies and ensures low coupling. The observed object does not depend on any concrete observer implementations, only on the observer interface, which keeps the design extensible.

Passing a DTO to observers preserves encapsulation of the observed object. Observers receive a snapshot of the current state instead of direct access to internal model objects, which prevents unintended modifications and aligns with the DTO principles discussed in the literature.

Observer registration is handled by the controller, which serves as a coordination point between view and model layers. This avoids direct dependencies between view and model while maintaining a clear ownership of observer lifecycle management. The chosen design maintains high cohesion and avoids introducing a spider-in-the-web controller.

The Observer pattern therefore improves responsiveness, reduces coupling, and clarifies responsibility boundaries without introducing architectural violations.

## 4.3 Evaluation of Additional Design Patterns

In addition to the Observer pattern, three additional GoF patterns were introduced: Singleton, Strategy, and Factory. Each pattern addresses a concrete design problem and is properly integrated into the system architecture.

The Singleton pattern is used for shared utility tools such as logging and database handling. These classes represent unique resources that must not be duplicated. Using the implementation accidental instantiation can be avoided while ensuring consistent access without exposing global state. The use of Singleton is limited to several cases and does not appear in business or domain objects, avoiding common misuse of the pattern.

The Strategy pattern is applied to pricing calculation, which depends on customer membership and previous repair informations. Encapsulating pricing logic into separate strategy classes removes conditional logic from the repair order and improves cohesion. New pricing rules can be introduced without modifying existing business logic, which directly supports the Open-Closed Principle described in Chapter 9.

The Factory pattern is used to centralize object instantiation logic for data handlers and pricing strategies. This allows also dynamically initiation of an arbitrary class at runtime, as well as simplifying unit testing by isolating creation concerns.

All three patterns are applied in suitable code sections only where they provide clear benefits. None of them overlap in responsibility, and each addresses a different form of variation or lifecycle management. Their usage is explicitly motivated and consistent with the intent of the original GoF definitions.

### 4.4 Architectural Considerations and Overall Quality

The solution maintains a clear layered MVC architecture throughout the implementation. Dependencies follow the intended direction from view to controller to model and integration layers. Exceptions and observer notifications cross layer boundaries in a controlled and well-defined manner.

Encapsulation is preserved by strict use of DTOs at architectural layer boundaries, interfaces for polymorphic behavior, and private state within domain objects. Coupling is reduced through observer interfaces, strategy delegation, and factory-based instantiation. Cohesion is improved by separating distinct responsibilities into focused classes.

One future improvement would be to further decouple observer registration from controller initialization, possibly through a dedicated configuration class. However, within the scope of this seminar, the current design represents a reasonable balance between simplicity and architectural correctness.

In conclusion, the implemented solution fulfills all requirements for Seminar 4, adheres closely to course literature guidelines, and demonstrates a solid understanding of exception handling and design pattern application in an object-oriented system.

## References

[1] Leif Lindbäck, *A First Course in Object-Oriented Development — A Hands-On Approach*, Course material for IV1350, KTH Royal Institute of Technology, January 14, 2026.

[2] Mo Wang, "ElectricBikeRepair — Source code for Seminar 4," 2026. [Online]. Available: `https://github.com/mo-mosverkstad/java-learning/tree/main/maven-projects/ElectricBikeRepair/`